EVALUACIÓN TÉCNICA

Desarrollador Full Stack

Prueba práctica + arquitectura + entrevista técnica

Candidato	Diego Alejandro Arismendi Guzmán
Fecha	03/11/2026
Entrevistador (solo para el evaluador)	
Nivel objetivo	Full Stack Intermedio – Senior

Objetivo

Medir capacidad real para analizar código, diseñar soluciones, desarrollar funcionalidades, trabajar con datos, asegurar una aplicación, diagnosticar fallas y operar software en producción.

Cómo utilizar este documento

1. Duración sugerida de la prueba escrita/práctica: 150 a 180 minutos.
2. Duración sugerida de la entrevista técnica: 45 a 60 minutos.
3. Documentación oficial: permitida. No se permite el uso de asistentes de IA durante la prueba.
4. No se evalúa únicamente que la respuesta funcione; se evalúan seguridad, mantenibilidad, rendimiento, pruebas, observabilidad y calidad de las decisiones.
5. Los fragmentos están escritos principalmente en TypeScript/JavaScript, React, Node.js y SQL porque permiten evaluar conceptos transferibles a otros stacks.

Componente	Puntos	Qué mide
1. Revisión y optimización de código	25	Calidad, rendimiento, seguridad, frontend, backend y SQL.
2. Caso de arquitectura	25	Diseño, escalabilidad, disponibilidad, datos, integración y trade-offs.
3. Implementación backend + datos	20	API, transacciones, concurrencia, idempotencia y modelado.
4. Frontend y experiencia	10	Estado, asincronía, UX, accesibilidad, rendimiento y manejo de errores.
5. Testing, seguridad y diagnóstico	10	Estrategia de pruebas, OWASP, debugging y resiliencia.
6. DevOps y operación	5	CI/CD, despliegue, rollback, observabilidad y secretos.
7. Comunicación técnica	5	Claridad, supuestos y justificación de decisiones.

PARTE I — PRUEBA PARA EL CANDIDATO

1. Revisión y optimización de código — 25 puntos

Para cada fragmento: **identifique problemas, explique el impacto y proponga una versión mejorada o una estrategia de corrección.**

1.1 Backend / API — 10 puntos

```
app.get('/api/orders', async (req, res) => {
  const userId = req.query.userId;

  const orders = await db.query(
    `SELECT * FROM orders WHERE user_id = ${userId}`
  );

  for (const order of orders.rows) {
    const items = await db.query(
      `SELECT * FROM order_items WHERE order_id = ${order.id}`
    );
    order.items = items.rows;
  }

  res.status(200).json(orders.rows);
});
```

1. Encuentre al menos 8 problemas o riesgos.

    - Envío del ID del usuario mediante la URL: El endpoint obtiene el identificador del usuario mediante req.query.userId, permitiendo que el cliente pueda modificar fácilmente este parámetro. Esto representa un riesgo porque una persona podría intentar consultar los pedidos de otro usuario sin autorización. Como consecuencia, podría producirse una exposición de información privada y una vulnerabilidad conocida como BOLA.

- Vulnerabilidad SQL Injection: se utiliza interpolación de variables directamente dentro de las consultas SQL, incorporando datos externos sin parametrización. Esto es peligroso porque un atacante podría manipular los valores enviados para alterar la estructura de la consulta. Como consecuencia, podría obtener acceso no autorizado, modificar información o comprometer la base de datos.
- Ausencia de autenticación: No evidencia en el código proporcionado ningún mecanismo para verificar la identidad de quien realiza la solicitud. Esto significa que no existe una comprobación visible de que la persona que intenta consultar los pedidos haya iniciado sesión. Como consecuencia, usuarios no autenticados podrían intentar acceder a información privada.
- Ausencia de autorización: El código no comprueba si el usuario que realiza la solicitud tiene permisos para consultar los pedidos solicitados. Aunque una persona esté autenticada, podría intentar acceder a información perteneciente a otro usuario. Como consecuencia, se podría producir un acceso indebido a información privada de terceros.
- Problema N+1 en las consultas: Se ejecuta una consulta adicional a la base de datos por cada pedido encontrado, dentro de un ciclo for. Esto provoca que el número de consultas aumente proporcionalmente a la cantidad de pedidos. Como consecuencia, se incrementan los tiempos de respuesta, el consumo de recursos y la carga sobre la base de datos.
- Utilización de SELECT *: Las consultas utilizan SELECT * para recuperar todas las columnas de las tablas, sin importar si son necesarias para la respuesta. Esto puede generar transferencia innecesaria de información y recuperación de datos que el frontend no necesita. Como consecuencia, aumenta el consumo de memoria, se reduce la eficiencia y existe el riesgo de exponer información sensible.
- Ausencia de paginación: Devuelve todos los pedidos encontrados sin establecer límites en la cantidad de registros. Esto puede resultar ineficiente cuando un usuario tiene un número elevado de pedidos. Como consecuencia, pueden generarse respuestas demasiado grandes, mayor consumo de memoria y tiempos de respuesta elevados.
- Ausencia de manejo de errores: El código no implementa un mecanismo para capturar y gestionar posibles excepciones durante las consultas a la base de datos. Esto significa que, si ocurre un fallo de conexión o una consulta presenta un error, no existe un tratamiento controlado dentro del endpoint. Como consecuencia, pueden generarse errores inesperados, respuestas inadecuadas y dificultades para identificar y solucionar fallos.
- Ausencia de límites y controles de consumo: No evidencia mecanismos para limitar la cantidad de solicitudes ni controlar el volumen de información consultada. Esto podría permitir que un cliente realice numerosas solicitudes en poco tiempo o intente consumir grandes cantidades de recursos. Como consecuencia, puede producirse una sobrecarga del servidor, degradación del rendimiento e incluso indisponibilidad del servicio.

2. Indique cuáles son de seguridad, rendimiento, mantenibilidad y diseño de API.
   Seguridad:
   - Envío del ID del usuario mediante la URL
   - Vulnerabilidad SQL Injection
   - No hay autenticación
   - No hay autorización sobre los pedidos

   Rendimiento:
   - Problema N+1 en las consultas
   - No implementa paginación

   Rendimiento/mantenibilidad:
   - Utiliza SELECT *
   - Ausencia de límites y controles de consumo

   Diseño de API:
   - Ausencia de manejo de errores

3. Proponga una versión mejorada del endpoint o pseudocódigo equivalente.
   R: 1\1.1\endpoint.js

4. Explique qué índices o ajustes de base de datos consideraría.
   - Índice para buscar pedidos por usuario: se propone crear un índice sobre user_id en la tabla orders. Esto permite localizar más rápidamente los pedidos asociados a un usuario, evitando búsquedas innecesarias sobre toda la tabla.
   - Índice para buscar artículos por pedido: se agrega un índice sobre order_id en la tabla order_items. Esto

facilita recuperar los artículos relacionados con cada pedido y mejora el rendimiento de las consultas y relaciones entre tablas.

- Índice compuesto para filtrar y ordenar: se propone un índice compuesto utilizando user_id, created_at e id. Su objetivo es optimizar la consulta que filtra los pedidos por usuario y los organiza desde el más reciente hasta el más antiguo.
- Claves foráneas: se establecen relaciones entre las tablas orders, users y order_items. Esto garantiza que cada pedido esté asociado a un usuario existente y que cada artículo pertenezca a un pedido válido, evitando registros huérfanos.
- Restricciones de integridad: se implementan restricciones para impedir que existan pedidos sin usuario asociado o artículos sin pedido. También se validan las cantidades y los precios para evitar valores inconsistentes, como cantidades negativas o precios inferiores a cero.
- Análisis del rendimiento: se propone utilizar herramientas de análisis de PostgreSQL para revisar cómo se ejecutan las consultas, identificar operaciones lentas y comprobar si los índices están siendo utilizados correctamente.

1.2 Frontend / React — 8 puntos

```
function ProductList({ api }) {
  const [products, setProducts] = useState([]);
  const [search, setSearch] = useState('');

  useEffect(() => {
    api.get('/products?search=' + search).then((response) => {
      products.push(...response.data);
      setProducts(products);
    });
  }, [search, products]);

  return (
    <div>
      <input value={search} onChange={(e) => setSearch(e.target.value)} />
      {products.map((p, index) => (
        <Product key={index} data={p} onClick={() => console.log(p)} />
      ))}
    </div>
  );
}
```

5. Identifique al menos 6 problemas.
   - Mutación directa del estado: se utiliza products.push(...response.data) para agregar productos directamente al estado existente. Problema: React recomienda no modificar directamente los estados, porque puede generar comportamientos inesperados y dificultar la detección de cambios.
   - Dependencias incorrectas en useEffect: el arreglo de dependencias contiene [search, products]. Problema: cada vez que se actualizan los productos, se vuelve a ejecutar el efecto y se genera otra solicitud al servidor. Esto puede producir un ciclo de solicitudes innecesarias.
   - Ausencia de debounce: la solicitud se ejecuta cada vez que cambia el contenido del campo de búsqueda. Problema: si el usuario escribe una palabra de diez caracteres, puede generar hasta diez solicitudes consecutivas.
   - No se manejan los errores: la solicitud utiliza .then() sin implementar un mecanismo para capturar errores. Problema: si el servidor falla, no hay una respuesta controlada ni una forma de informar al usuario.
   - No existe indicador de carga: el componente no tiene un estado loading. Problema: el usuario no sabe si los productos están siendo consultados o si la aplicación está procesando la búsqueda.
   - No se controlan las respuestas fuera de orden: no existe ningún mecanismo para verificar si una respuesta corresponde a la búsqueda más reciente. Problema: una solicitud anterior puede tardar más y sobrescribir los resultados de una búsqueda posterior, mostrando información incorrecta.
   - No existe cancelación de solicitudes: el componente no cancela las solicitudes pendientes cuando cambia el término de búsqueda o se desmonta. Problema: se desperdician recursos y pueden producirse actualizaciones de estado que ya no son necesarias.
   - Uso del índice como key: se utiliza key={index} para identificar cada producto. Problema: si los resultados cambian de orden, se agregan o eliminan elementos, React puede asociar incorrectamente los componentes

con sus datos. Es preferible utilizar un identificador único del producto.

- Falta de manejo de resultados vacíos: no existe una condición para mostrar un mensaje cuando la búsqueda no encuentra productos. Problema: el usuario puede interpretar que la aplicación no respondió correctamente.

6. Explique cómo evitaría solicitudes excesivas mientras el usuario escribe.
   R: Implementaría una técnica llamada debounce.
   Consiste en esperar un tiempo determinado después de que el usuario deja de escribir antes de realizar la solicitudservidor.Por ejemplo, establecería un tiempo de espera de 400 milisegundos. Si el usuario continúa escribiendo, el contador se reinicia y la solicitud solo se ejecuta cuando deja de escribir.

7. Explique cómo manejaría respuestas que llegan fuera de orden, loading, error y cancelación.
   Implementaría cuatro mecanismos:
   - Respuestas fuera de orden: utilizaría una bandera de control para garantizar que solamente la solicitud correspondiente a la búsqueda más reciente pueda actualizar los resultados. Así se evita que una respuesta antigua sobrescriba una nueva.
   - Loading: crearía un estado de carga para informar al usuario cuándo se están consultando los productos. Se activa al iniciar la solicitud y se desactiva cuando termina.
   - Error: implementaría un bloque catch para capturar errores de conexión o respuestas fallidas del servidor y mostrar un mensaje comprensible.
   - Cancelación: utilizaría AbortController para cancelar las solicitudes anteriores cuando cambia el término de búsqueda o cuando el componente se desmonta. Esto evita solicitudes innecesarias y consumo de recursos.
8. Proponga el código o pseudocódigo corregido.
   R: 1\1.2\Ejercicio_1.2_Optimizacion_ProductList.jsx

1.3 SQL / datos — 7 puntos

```
SELECT c.name,
       COUNT(o.id) AS orders,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM customers c
JOIN orders o ON o.customer_id = c.id
JOIN order_items oi ON oi.order_id = o.id
WHERE DATE(o.created_at) = '2026-10-01'
GROUP BY c.name
ORDER BY revenue DESC;
```

9. ¿Qué problemas de exactitud o rendimiento podrían aparecer?
   - Uso de DATE() sobre la columna de fecha: aplicar esta función sobre created_at puede impedir que PostgreSQL aproveche eficientemente un índice convencional sobre esa columna, generando consultas más lentas.
   - Duplicación en el conteo de pedidos: al relacionar orders con order_items, un pedido puede aparecer varias veces porque puede contener varios artículos. Por esta razón, COUNT(o.id) podría contar el mismo pedido más de una vez y producir resultados incorrectos.
   - Agrupación únicamente por nombre del cliente: pueden existir diferentes clientes con el mismo nombre, lo que provocaría que sus pedidos e ingresos se mezclen en un solo resultado. Se recomienda agrupar también por el identificador único del cliente.
   - Posibles valores NULL: si no existen valores para sumar, SUM() puede devolver NULL, lo que puede dificultar la interpretación de los ingresos.
   - Alto volumen de registros: las uniones, agrupaciones y ordenamientos pueden consumir muchos recursos cuando las tablas contienen millones de registros, aumentando los tiempos de respuesta y afectando el rendimiento general de la base de datos.

10. Reescriba la consulta para filtrar el día sin aplicar una función sobre la columna de fecha.
    R: 1\1.3\Consulta_Optimizada_Ventas_por_Cliente.sql
11. Proponga los índices que consideraría y explique por qué.
    - Índice sobre la fecha de los pedidos: columna orders.created_at. Permite localizar más rápidamente los pedidos correspondientes al día consultado, evitando recorrer innecesariamente toda la tabla.
    - Índice compuesto sobre fecha y cliente: columnas orders(created_at, customer_id). Facilita filtrar los pedidos

por fecha y disponer de la información del cliente asociado. Es especialmente útil para esta consulta, que utiliza ambas columnas.

- Índice sobre los artículos del pedido: columna order_items.order_id. Mejora la búsqueda de los artículos relacionados con cada pedido y facilita la operación JOIN entre orders y order_items.
- Claves primarias e índices existentes: verificaría que customers.id, orders.id y order_items.id tengan sus respectivas claves primarias. Estas normalmente crean índices automáticamente y facilitan las relaciones entre las tablas.

12. ¿Qué haría si la tabla de orders tuviera cientos de millones de registros?
    - Particionamiento de la tabla: dividiría la tabla orders en particiones por rangos de fechas, por ejemplo, una partición por mes o por año. Esto permitiría que PostgreSQL consulte únicamente las particiones correspondientes al período solicitado, reduciendo la cantidad de registros que necesita examinar.
    - Índices optimizados: revisaría los índices de cada partición y priorizaría los que respondan a las consultas más frecuentes. También evaluaría índices BRIN sobre created_at cuando los registros estén almacenados en un orden correlacionado con la fecha, porque pueden ocupar mucho menos espacio que un índice B-tree tradicional.
    - Resúmenes y agregaciones precalculadas: implementaría tablas de resumen o vistas materializadas con información de ventas agrupada por cliente y día. De esta manera, en lugar de calcular todos los ingresos recorriendo millones de registros cada vez, se podrían consultar resultados previamente calculados y actualizados periódicamente.
    - Optimización y mantenimiento: utilizaría herramientas como EXPLAIN ANALYZE para identificar consultas costosas, revisaría las estadísticas de la base de datos y establecería tareas de mantenimiento como VACUUM y ANALYZE.
    - Escalabilidad de infraestructura: si el volumen de consultas lo requiere, evaluaría réplicas de lectura para distribuir las consultas de reportes y análisis, evitando que compitan directamente con las operaciones de escritura del sistema.

2. Caso de arquitectura — 25 puntos

> **Caso.** Una empresa lanzará “CommerceHub”, una plataforma web y móvil para vender productos de múltiples tiendas. Debe soportar catálogo, inventario, carrito, órdenes, pagos, imágenes, notificaciones y trazabilidad administrativa.

Condición	Requisito
Usuarios	200.000 registrados; picos de 8.000 usuarios concurrentes en campañas.
Tráfico	Hasta 1.000 solicitudes por segundo en picos.
Inventario	No se debe vender una unidad inexistente. Debe soportar reserva temporal durante el pago.
Pagos	Proveedor externo. Puede responder lento y envía webhooks de confirmación.
Disponibilidad	Objetivo inicial 99,9 %. Recuperación ante fallas y backups verificables.
Canales	Web, app móvil y futuro portal administrativo.

Condición	Requisito
Seguridad	Autenticación, roles, auditoría, protección de datos y secretos.
Operación	Logs, métricas, trazas, alertas, CI/CD y rollback.

Entregable esperado:

13. Dibuje la arquitectura de alto nivel. Puede usar bloques y flechas.

    R: 2\arquitectura de alto nivel.pptx

14. Explique el flujo completo desde que un usuario presiona “Pagar” hasta que la orden queda confirmada.
    El proceso debe garantizar que el inventario no se venda dos veces y que un pago no genere órdenes duplicadas.
    - el backend recibe la solicitud de pago, identifica al usuario, verifica los productos, cantidades, precios y disponibilidad real del inventario.
    - el sistema reserva las unidades solicitadas mediante una operación atómica en la base de datos, evitando que otro usuario pueda adquirir las mismas unidades.
    - se crea una orden con estado PENDING_PAYMENT, asociada a la reserva y al usuario. Se registra el tiempo de expiración de la reserva.
    - el backend solicita al proveedor externo la creación de una transacción de pago y utiliza una clave de idempotencia para evitar operaciones duplicadas.
    - si el proveedor responde, el usuario continúa con el proceso de pago. Si tarda demasiado, la orden permanece pendiente hasta recibir una confirmación o completar una verificación.
    - el proveedor envía una notificación al backend indicando si el pago fue aprobado, rechazado o continúa pendiente. El backend verifica la autenticidad del mensaje.
    - si el pago fue aprobado y la reserva sigue siendo válida, se confirma la orden, se descuenta definitivamente el inventario y se registra el pago.
    - se envía la confirmación por correo, aplicación o WhatsApp mediante una cola de mensajes.

    Si el pago es rechazado o la reserva expira sin pago confirmado, se libera el inventario y la orden se marca como cancelada o fallida, según corresponda.

15. Defina componentes principales: frontend, backend, autenticación, datos, caché, mensajería, almacenamiento de imágenes y observabilidad.
    - Frontend: aplicación web y móvil para consultar productos, administrar el carrito, realizar compras y consultar el estado de las órdenes. Se desarrollaría de manera independiente del backend para permitir incorporar posteriormente un portal administrativo.
    - Backend: aplicación organizada en módulos funcionales: usuarios, autenticación, catálogo, inventario, reservas, órdenes, pagos, notificaciones y auditoría. Centraliza las reglas de negocio y la comunicación con la base de datos y los servicios externos.
    - Autenticación y autorización: utilizaría autenticación mediante tokens seguros, con roles y permisos para clientes, administradores y operadores. Cada usuario solo podrá consultar o modificar los recursos que tenga autorizados.
    - Base de datos: PostgreSQL sería la fuente principal de información. Almacenaría usuarios, productos, tiendas, inventario, reservas, órdenes, pagos y registros administrativos. Utilizaría transacciones para proteger las operaciones críticas.
    - Caché: Redis permitiría almacenar temporalmente información de consulta frecuente, como catálogos, categorías y datos de sesiones. No utilizaría la caché como fuente definitiva para confirmar existencias.
    - Mensajería: implementaría una cola de mensajes para procesar notificaciones, correos, actualizaciones y tareas que no necesitan ejecutarse durante la solicitud del usuario. Esto evita bloquear el backend mientras se realizan procesos secundarios.
    - Almacenamiento de imágenes: utilizaría almacenamiento de objetos para fotografías de productos y archivos, acompañado de una CDN para entregar estos recursos rápidamente.
    - Observabilidad: implementaría logs centralizados, métricas, trazas y alertas para monitorear errores, tiempos de respuesta, disponibilidad, uso de recursos y comportamiento de los pagos.
    - Seguridad y auditoría: registraría las operaciones administrativas y eventos importantes, protegería las

comunicaciones mediante HTTPS y almacenaría las credenciales y secretos en un servicio seguro, sin incluirlos en el código fuente.

16. Indique qué información requiere consistencia fuerte y qué procesos pueden ser eventualmente consistentes.
    Consistencia fuerte
    Se aplicaría en las operaciones críticas del negocio, donde es indispensable que la información esté actualizada y sea confiable en todo momento. Esto incluye el inventario, cuyas unidades disponibles deben actualizarse de forma atómica para evitar vender productos inexistentes; las reservas, que deben tener estados y tiempos de vigencia definidos para impedir que varios usuarios reserven las mismas unidades; las órdenes, cuyos estados de creación, confirmación y cancelación deben mantenerse correctamente; los pagos, garantizando que una transacción aprobada no genere cobros ni confirmaciones duplicadas; y los usuarios y permisos, cuyas modificaciones deben aplicarse de forma segura para proteger la información y controlar los accesos.
    Consistencia eventual
    Se aplicaría en los procesos que no requieren una actualización inmediata y que pueden sincronizarse progresivamente sin afectar las operaciones principales del negocio. Esto incluye las notificaciones por correo, mensajes y avisos, que pueden enviarse después de confirmar una operación; el catálogo y la caché, cuyas modificaciones pueden reflejarse progresivamente; los reportes y estadísticas, que pueden actualizarse mediante procesos en segundo plano; los índices de búsqueda, cuya información puede sincronizarse posteriormente; y las imágenes y contenidos multimedia, que pueden procesarse y distribuirse después de guardar la información principal del producto.

17. Explique cómo evitaría overselling, cobros duplicados y procesamiento duplicado de webhooks.

    - implementaría reservas temporales de inventario y transacciones en PostgreSQL. Antes de reservar una cantidad, el sistema verifica que exista disponibilidad y realiza la actualización de manera atómica. También establecería una fecha de expiración para liberar automáticamente las reservas que no completen el pago.
    - utilizaría claves de idempotencia para identificar cada intento de pago. Si el usuario presiona varias veces el botón de pagar o se repite una solicitud por problemas de conexión, el backend reconoce la operación existente y evita crear un nuevo cobro.
    - cada notificación recibida del proveedor de pagos debe tener un identificador único. Lo almacenaría en una tabla de eventos procesados con una restricción de unicidad. Si llega nuevamente el mismo evento, el sistema identifica que ya fue procesado y no repite la operación.
    - validaría la firma digital de cada webhook y, cuando exista alguna duda sobre su resultado, consultaría directamente el estado de la transacción con el proveedor.
    - implementaría una máquina de estados para las órdenes y los pagos, evitando transiciones incorrectas. Por ejemplo, una orden ya confirmada no debe volver a confirmarse ni generar un segundo descuento de inventario.

18. Proponga una estrategia para escalar y para recuperarse de fallas.
    Implementaría una estrategia de escalabilidad horizontal mediante varias instancias del backend detrás de un balanceador de carga, permitiendo agregar automáticamente nuevas instancias cuando aumente el tráfico. Para optimizar el rendimiento, utilizaría índices adecuados, paginación, consultas optimizadas y sistemas de caché que reduzcan la carga sobre PostgreSQL.
    En cuanto a la base de datos, utilizaría una instancia principal para las operaciones de escritura y réplicas de lectura destinadas a consultas de catálogo y reportes, garantizando que las operaciones críticas de inventario y pagos siempre se validen contra la base de datos principal. También implementaría procesamiento asíncrono mediante colas de mensajes para distribuir notificaciones y tareas secundarias, incorporando reintentos controlados y una cola de mensajes fallidos (Dead Letter Queue) para gestionar los procesos que no puedan completarse.
    Para garantizar la disponibilidad, configuraría una infraestructura con redundancia entre zonas de disponibilidad y mecanismos de recuperación ante fallas. Además, establecería copias de seguridad automáticas, recuperación a un punto específico en el tiempo y pruebas periódicas de restauración para asegurar que la información pueda recuperarse correctamente.
    Finalmente, implementaría monitoreo y alertas para detectar errores en los pagos, agotamiento de conexiones, consultas lentas, uso elevado de recursos, saturación de colas y caídas de servicios. Complementaría esta estrategia con integración y despliegue continuo (CI/CD), pruebas automatizadas y despliegues progresivos, permitiendo regresar a una versión estable anterior mediante un proceso de rollback si se presentan errores.

19. Explique qué desplegaría inicialmente como monolito modular y qué justificaría separar en servicios más adelante.

comenzaría con un monolito modular, una sola aplicación backend organizada en diferentes módulos

Inicialmente integraría los módulos de usuarios y autenticación, catálogo, inventario, órdenes, pagos, notificaciones y auditoría. De esta manera, tendría toda la lógica del negocio centralizada, pero organizada para facilitar su mantenimiento, desarrollo y futuras modificaciones.

Considero que esta es la mejor opción para iniciar porque permite desarrollar la plataforma sin agregar demasiada complejidad desde el principio. Además, facilita controlar las operaciones principales y mantener una comunicación directa entre los diferentes módulos.

A medida que la plataforma crezca, evaluaría separar algunos módulos en servicios independientes, dependiendo de las necesidades reales. Por ejemplo, separaría notificaciones si aumenta considerablemente el volumen de mensajes; pagos, si se requieren más integraciones y controles especializados; catálogo y búsqueda, si aumenta el número de consultas; e inventario, si la operación de múltiples tiendas exige una gestión más compleja.

Mi decisión sería comenzar con una arquitectura sencilla, organizada y preparada para crecer. No implementaría microservicios desde el inicio, sino que los incorporaría progresivamente cuando el volumen de usuarios, las necesidades de rendimiento o la complejidad del negocio realmente lo justifiquen.

20. Seleccione una nube o una arquitectura agnóstica y justifique 5 decisiones técnicas importantes.

    Para implementar este proyecto seleccionaría AWS, utilizando servicios administrados que permitan escalar la plataforma, reducir la carga operativa y garantizar su disponibilidad. Las cinco decisiones técnicas principales serían:

    - Backend desplegado en Amazon ECS con Fargate que permitiría ejecutar varias instancias del monolito modular sin tener que administrar directamente los servidores. Además, facilitaría el escalamiento horizontal cuando aumente el tráfico de usuarios.
    - PostgreSQL administrado mediante Amazon RDS utilizaría una base de datos relacional para garantizar transacciones seguras, integridad de los datos y consistencia en las operaciones de inventario, órdenes y pagos. También configuraría alta disponibilidad y respaldos automáticos.
    - Redis mediante Amazon ElastiCache: permitiría almacenar información consultada frecuentemente, reducir las solicitudes a la base de datos y gestionar información temporal. Sin embargo, las existencias definitivas seguirían controlándose en PostgreSQL.
    - Amazon SQS para mensajería: utilizaría colas para procesar notificaciones y tareas secundarias de manera asíncrona. Esto permitiría desacoplar procesos, implementar reintentos y evitar que las operaciones de compra dependan de tareas que no son críticas.
    - Amazon S3, CloudFront y CloudWatch: utilizaría S3 para almacenar imágenes, CloudFront para distribuirlas rápidamente y CloudWatch para centralizar métricas, registros y alertas. Complementaría la solución con integración y despliegue continuo, respaldos y mecanismos de reversión en caso de fallas.

> **Importante.** No se otorgarán puntos adicionales por usar microservicios, Kubernetes o tecnologías complejas si no existe una razón técnica. Se evalúa la calidad del razonamiento y los trade-offs.

3. Implementación backend + datos — 20 puntos

Diseñe el endpoint **`POST /api/v1/orders`** para crear una orden a partir de un carrito.

```
Request
{
  "customerId": "C-1001",
  "items": [
    { "sku": "SKU-001", "quantity": 2 },
    { "sku": "SKU-050", "quantity": 1 }
  ]
}

Header
Idempotency-Key: 8bd95f8b-...
```

21. Defina las validaciones de entrada.

    Antes de procesar la orden, realizaría las siguientes validaciones:

    - verificar que el cliente esté autenticado y que tenga permiso para realizar la compra. El customerId recibido debe corresponder al usuario autenticado; no confiaría únicamente en el valor enviado por el cliente.

- comprobar que el cuerpo de la petición sea un JSON válido y que contenga customerId e items.
- validar que customerId sea obligatorio, tenga el formato esperado y corresponda a un cliente existente y activo.
- verificar que items sea un arreglo no vacío y que cada elemento contenga un SKU válido y una cantidad entera mayor que cero.
- evitar que un mismo SKU aparezca varias veces en la solicitud o consolidar sus cantidades antes de procesar la compra.
- comprobar que todos los SKU correspondan a productos registrados y disponibles para la venta.
- validar que exista stock suficiente para cada producto, sin depender únicamente de una consulta previa, ya que otro cliente podría comprar esas unidades simultáneamente.
- exigir el encabezado Idempotency-Key, validar su formato y comprobar si ya existe una solicitud procesada con esa misma clave.

22. Escriba código, pseudocódigo o una secuencia SQL que garantice que el stock no quede negativo aun cuando dos clientes compren simultáneamente la última unidad.

    para garantizar que dos clientes no puedan comprar simultáneamente la última unidad disponible, utilizaría una actualización atómica del inventario dentro de una transacción de PostgreSQL.

    La operación debe descontar las unidades únicamente cuando exista stock suficiente.

    ```
    UPDATE inventory
    SET available_quantity = available_quantity - 2
    WHERE sku = 'SKU-001'
    AND available_quantity >= 2;
    ```

    después de ejecutar la consulta, comprobaría cuántas filas fueron actualizadas. Si no se actualizó ninguna, significa que no existe inventario suficiente y la compra no puede continuar.

    Para procesar varios productos, realizaría las actualizaciones dentro de una misma transacción. Si alguno de los productos no tiene unidades disponibles, ejecutaría un ROLLBACK para revertir todos los descuentos y evitar que la orden quede parcialmente procesada.

23. Explique dónde comienza y termina la transacción.
    la transacción comenzaría cuando el backend inicie el procesamiento definitivo de la orden en PostgreSQL. Dentro de ella realizaría las siguientes operaciones:

    - Inicio: ejecutar BEGIN.
    - Idempotencia: registrar o reclamar la clave de idempotencia para evitar que otra solicitud procese la misma compra.
    - Validación del inventario: consultar y reservar las unidades disponibles mediante actualizaciones atómicas.
    - Creación de la orden: insertar el registro en la tabla orders con estado PENDING_PAYMENT.
    - Registro de productos: insertar cada producto y su cantidad en order_items.
    - Registro de la reserva: guardar la información de las unidades reservadas y su fecha de expiración.
    - Finalización: ejecutar COMMIT si todas las operaciones fueron exitosas o ROLLBACK si ocurre algún error.

    La transacción terminaría antes de comunicarse con el proveedor externo de pagos. De esta manera, no mantendría bloqueos ni conexiones de base de datos ocupadas mientras espero la respuesta del proveedor.

24. Diseñe un mecanismo de idempotencia para que reintentar la misma petición no cree dos órdenes.
    implementaría un mecanismo basado en el encabezado Idempotency-Key, que identifica de manera única cada intento lógico de creación de una orden. El backend recibiría la clave enviada por el cliente y la asociaría al usuario autenticado. Posteriormente, consultaría la tabla idempotency_keys para determinar si esa clave ya fue utilizada.

    Para controlar las solicitudes simultáneas, establecería una restricción UNIQUE sobre la combinación de cliente y clave, evitando que dos solicitudes con la misma clave puedan procesar dos órdenes diferentes. También almacenaría una huella (hash) del contenido de la solicitud, de manera que, si se intenta reutilizar la misma clave con datos diferentes, el sistema responda con un conflicto.

    Si la solicitud ya fue procesada, el backend devolvería la misma orden y el resultado previamente registrado, sin volver a descontar el inventario. Si otra solicitud con la misma clave todavía se encuentra en proceso, respondería indicando que la operación sigue en curso, sin crear una segunda orden.

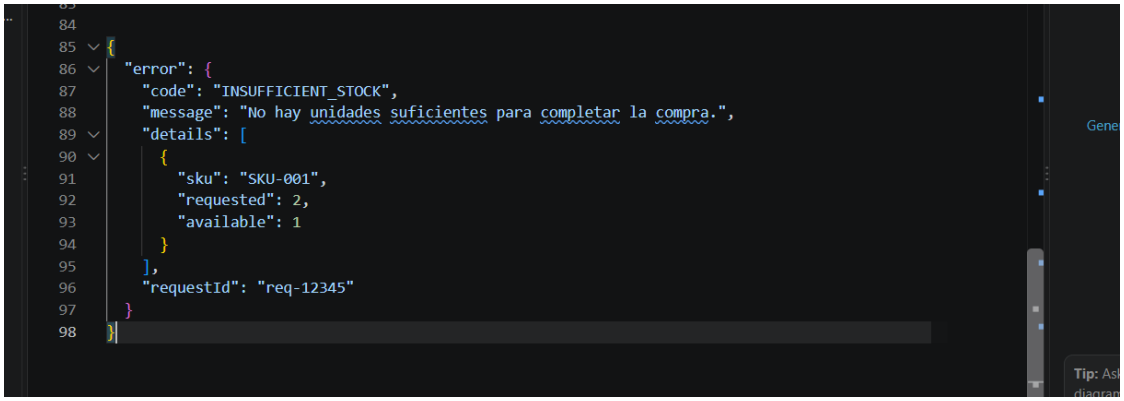
    Finalmente, conservaría la clave junto con el identificador de la orden, el estado del procesamiento y la respuesta correspondiente. Esto permitiría que, ante un problema de conexión, el cliente pueda repetir la petición sin generar compras duplicadas.

25. Indique códigos HTTP y estructura de errores para: datos inválidos, falta de stock, recurso inexistente, conflicto e error interno.

    Utilizaría códigos HTTP que permitan identificar claramente el resultado de cada operación.

Código HTTP	Situación	Descripción
201 Created	Orden creada	La orden fue creada correctamente y quedó pendiente de pago.
400 Bad Request	Datos inválidos	La solicitud tiene campos faltantes, cantidades incorrectas o un formato no válido.
401 Unauthorized	No autenticado	El cliente no ha iniciado sesión o su credencial no es válida.
404 Not Found	Recurso inexistente	El cliente o alguno de los productos solicitados no existe.
409 Conflict	Conflicto	No hay stock suficiente, la clave de idempotencia se reutilizó con datos diferentes o existe un conflicto de estado.
500 Internal Server Error	Error interno	Ocurrió un error inesperado durante el procesamiento.

    Mantendría una estructura de respuesta uniforme para los errores, por ejemplo:



```
{
  "error": {
    "code": "INSUFFICIENT_STOCK",
    "message": "No hay unidades suficientes para completar la compra.",
    "details": [
      {
        "sku": "SKU-001",
        "requested": 2,
        "available": 1
      }
    ],
    "requestId": "req-12345"
  }
}
```

    En caso de falta de stock, utilizaría el código 409 Conflict. Para errores de validación, incluiría los campos que deben corregirse. Los errores internos no expondrían información sensible, consultas SQL ni detalles técnicos de la infraestructura.

26. Proponga un modelo mínimo de tablas para orders, order_items, inventory e idempotency_keys/reservations.

```sql
-- 1. ORDENES
CREATE TABLE orders (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    customer_id VARCHAR(50) NOT NULL,
    status VARCHAR(30) NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    PRIMARY KEY (id),
    INDEX idx_orders_customer_created (customer_id, created_at),

    CONSTRAINT fk_orders_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id),

    CONSTRAINT chk_orders_status CHECK (
        status IN (
            'PENDING_PAYMENT',
            'PAID',
            'PAYMENT_FAILED',
            'CANCELLED',
            'EXPIRED'
        )
    ),

    CONSTRAINT chk_orders_total CHECK (total_amount >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```sql
-- 2. PRODUCTOS DE LAS ORDENES
CREATE TABLE order_items (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    order_id BIGINT UNSIGNED NOT NULL,
    sku VARCHAR(50) NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,

    PRIMARY KEY (id),
    INDEX idx_order_items_order (order_id),

    CONSTRAINT fk_order_items_order
        FOREIGN KEY (order_id) REFERENCES orders(id)
        ON DELETE CASCADE,

    CONSTRAINT chk_order_quantity CHECK (quantity > 0),
    CONSTRAINT chk_order_price CHECK (unit_price >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 3. INVENTARIO
CREATE TABLE inventory (
    sku VARCHAR(50) NOT NULL,
    available_quantity INT NOT NULL,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
        ON UPDATE CURRENT_TIMESTAMP,

    PRIMARY KEY (sku),

    CONSTRAINT chk_inventory_quantity
        CHECK (available_quantity >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

```sql
-- 4. RESERVAS DE INVENTARIO
CREATE TABLE inventory_reservations (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    order_id BIGINT UNSIGNED NOT NULL,
    sku VARCHAR(50) NOT NULL,
    quantity INT NOT NULL,
    status VARCHAR(20) NOT NULL,
    expires_at DATETIME NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,

    PRIMARY KEY (id),
    UNIQUE KEY uq_reservation_order_sku (order_id, sku),
    INDEX idx_reservations_expiration (status, expires_at),

    CONSTRAINT fk_reservations_order
        FOREIGN KEY (order_id) REFERENCES orders(id)
        ON DELETE CASCADE,

    CONSTRAINT fk_reservations_inventory
        FOREIGN KEY (sku) REFERENCES inventory(sku),

    CONSTRAINT chk_reservation_quantity CHECK (quantity > 0),

    CONSTRAINT chk_reservation_status CHECK (
        status IN ('ACTIVE', 'CONFIRMED', 'RELEASED', 'EXPIRED')
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

-- 5. CLAVES DE IDEMPOTENCIA
CREATE TABLE idempotency_keys (
    id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
    customer_id VARCHAR(50) NOT NULL,
    idempotency_key VARCHAR(255) NOT NULL,
    request_hash VARCHAR(64) NOT NULL,
    order_id BIGINT UNSIGNED NULL,
    status VARCHAR(20) NOT NULL,
    response_body JSON NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    expires_at DATETIME NULL,

    PRIMARY KEY (id),
    UNIQUE KEY uq_customer_idempotency (customer_id, idempote
    INDEX idx_idempotency_order (order_id),

    CONSTRAINT fk_idempotency_customer
        FOREIGN KEY (customer_id) REFERENCES customers(id),

    CONSTRAINT fk_idempotency_order
        FOREIGN KEY (order_id) REFERENCES orders(id),

    CONSTRAINT chk_idempotency_status CHECK (
        status IN ('PROCESSING', 'COMPLETED', 'FAILED')
    )
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

27. Explique cómo integraría el pago externo sin mantener una transacción de base de datos abierta mientras espera al proveedor.

    integraría el proveedor de pagos mediante un proceso separado de la transacción de inventario y creación de la orden. La idea es que la base de datos confirme primero la orden y la reserva de inventario, y que después el backend se comunique con el proveedor de pagos.

    El flujo sería el siguiente:

    - el backend valida la solicitud, reserva el inventario y crea la orden con estado PENDING_PAYMENT.
    - ejecuta COMMIT y finaliza la transacción. En este momento, las unidades quedan reservadas temporalmente, pero todavía no se consideran una venta definitiva.
    - fuera de la transacción, el backend envía la información necesaria al proveedor de pagos. Utilizaría una

clave de idempotencia para evitar que los reintentos generen cobros duplicados.

- si el pago es aprobado, el backend registra la confirmación y actualiza la orden a PAID. Si es rechazado, actualiza el estado correspondiente y libera la reserva.
- implementaría un endpoint para recibir las notificaciones del proveedor. Verificaría la firma de cada evento y evitaría procesar dos veces el mismo evento mediante un identificador único.
- si el proveedor no responde o la operación queda pendiente, mantendría la orden en PENDING_PAYMENT hasta verificar el resultado. No asumiría que un tiempo de espera significa que el pago fue rechazado.
- establecería un tiempo límite para completar el pago. Si vence y se confirma que no hubo un pago exitoso, liberaría las unidades reservadas mediante una operación transaccional.

Para evitar inconsistencias, todas las actualizaciones de estados de la orden, el pago y la reserva se realizarían mediante transacciones cortas e independientes. Si llega una confirmación de pago después de que la reserva haya expirado y el inventario ya se haya liberado, el sistema deberá gestionar el caso de forma controlada, por ejemplo, intentando reservar nuevamente las unidades o iniciando un reembolso.

mi enfoque sería mantener separadas las responsabilidades: PostgreSQL controla la integridad de las órdenes y del inventario; el proveedor externo procesa los pagos; y los Webhooks permiten sincronizar los resultados. Así se evitan transacciones prolongadas, descuentos duplicados de inventario y cobros repetidos, manteniendo la consistencia de la operación.

4. Frontend y experiencia de usuario — 10 puntos

> **Escenario.** La pantalla Checkout muestra dirección, productos, total y botón “Pagar”. El backend puede tardar entre 300 ms y 4 s; ocasionalmente la red móvil se interrumpe después de enviar la orden.

28. Describa los estados de UI que implementaría: idle, validando, enviando, confirmado, error recuperable, error definitivo u otros.
    implementaría estados que permitan controlar todo el proceso de compra: Idle, para revisar la información; Validando, para comprobar los datos; Enviando, para mostrar que la solicitud está en proceso; Pendiente de confirmación, cuando aún no hay respuesta del servidor; Confirmado, cuando la orden se crea correctamente; Error recuperable, para permitir reintentos seguros ante fallas de conexión; y Error definitivo, cuando se requiere corregir datos, solucionar falta de inventario o atender un rechazo del pago.
29. ¿Cómo evitaría doble clic/doble envío sin bloquear para siempre al usuario?
    para evitar que el usuario genere varias órdenes al presionar repetidamente el botón «Pagar», implementaría un control de estado en el frontend. Mientras la solicitud se encuentre en proceso, deshabilitaría temporalmente el botón y mostraría un indicador visual de carga.
    Además, utilizaría una clave de idempotencia (Idempotency-Key) generada antes de enviar la solicitud. Esta clave se mantendría durante los reintentos de una misma operación, permitiendo que el backend identifique las solicitudes repetidas y devuelva el resultado de la orden original.
    No bloquearía permanentemente al usuario. Si la solicitud tarda demasiado, mostraría un mensaje de espera y permitiría consultar el estado de la operación. Solo habilitaría una nueva compra cuando se conozca el resultado de la anterior o se haya resuelto su estado.
30. ¿Qué haría si el cliente pierde conexión justo después de enviar el POST y no sabe si la orden fue creada?
    Si el cliente pierde la conexión después de enviar el POST, no asumiría que la compra falló, ya que el backend podría haber creado la orden. Guardaría la clave de idempotencia para consultar el estado de la operación cuando se recupere la conexión y, si es necesario, reintentar la solicitud con la misma clave para evitar duplicados. Finalmente, actualizaría la interfaz según la respuesta real del servidor, informando al usuario si la orden fue confirmada, sigue pendiente o presentó un error.
31. Indique medidas mínimas de accesibilidad para formulario, errores y botón de pago.
    tomaría medidas de accesibilidad para que el proceso de compra sea fácil de utilizar para todas las personas y funcione correctamente en diferentes dispositivos. Cada campo del formulario tendría una etiqueta clara y asociada correctamente, mientras que los errores se mostrarían junto al campo correspondiente mediante mensajes descriptivos, sin depender únicamente de los colores. También permitiría navegar por todo el formulario utilizando el teclado y utilizaría colores con suficiente contraste para facilitar la lectura. Durante los tiempos de espera, mostraría indicadores de carga y mensajes comprensibles, apoyados en atributos como aria-live para que los

lectores de pantalla puedan anunciar los cambios. Finalmente, el botón de pago tendría un nombre claro, un estado accesible cuando esté deshabilitado y una indicación visible durante el procesamiento. Además, el diseño sería adaptable a celulares, tabletas y computadores.

32. Explique qué estado dejaría local, qué estado compartiría globalmente y qué datos trataría como “server state”.

Separaría el manejo de la información en tres tipos de estado para mantener el frontend organizado y evitar duplicaciones. El estado local controlaría los datos temporales del Checkout, como los campos del formulario, la dirección seleccionada, los errores y el indicador de carga. El estado global almacenaría información compartida entre diferentes pantallas, como el usuario autenticado, la sesión y el carrito de compras, utilizando herramientas como Context API, Redux Toolkit o Zustand. Por último, el estado del servidor incluiría productos, precios, inventario, órdenes y pagos, gestionados mediante herramientas como TanStack Query para mantener la información sincronizada con el backend. Esta separación facilitaría el mantenimiento, reduciría inconsistencias y ofrecería una experiencia de compra más confiable.

5. Testing, seguridad y diagnóstico — 10 punto

Situación	Respuesta solicitada
Pruebas	Defina 8 pruebas prioritarias para el flujo de creación de orden: unitarias, integración y/o E2E.
Seguridad	Enumere riesgos relevantes del OWASP Top 10 para esta solución y controles concretos.
Incidente	Producción reporta órdenes duplicadas solo cuando el proveedor de pago responde lento. ¿Cómo investigaría el problema?
Datos	Una migración cambia una columna usada por una versión anterior del backend. ¿Cómo desplegaría el cambio sin interrumpir servicio?

Pruebas

1.comprobaría que el sistema rechace solicitudes con campos obligatorios vacíos, cantidades negativas, SKU inválidos o formatos incorrectos. Sería una prueba unitaria.
2.verificaría que únicamente los usuarios autenticados y autorizados puedan crear órdenes y que no puedan realizar compras en nombre de otros clientes. Sería una prueba de integración.
3.comprobaría que el sistema permita comprar cuando existe suficiente stock y rechace la operación cuando las unidades disponibles sean insuficientes. Sería una prueba de integración.
4.simularía dos usuarios intentando comprar simultáneamente las últimas unidades de un producto para verificar que el inventario no quede en valores negativos ni se produzca una sobreventa. Sería una prueba de integración.
5.enviaría varias solicitudes con la misma clave de idempotencia para comprobar que el sistema cree una sola orden y devuelva el resultado de la operación original. Sería una prueba de integración.
6.simularía respuestas exitosas, rechazadas y demoradas del proveedor de pagos para verificar que las órdenes cambien correctamente de estado. Sería una prueba de integración.
7.interrumpiría la comunicación después de enviar la solicitud para comprobar que el sistema pueda recuperar el estado de la operación sin generar órdenes duplicadas. Sería una prueba E2E.
8. realizaría una compra desde el frontend hasta el backend, verificando la validación de datos, reserva de inventario, creación de la orden y confirmación del pago. Sería una prueba E2E.

Seguridad

Tendría en cuenta los riesgos más relevantes del OWASP Top 10 y aplicaría controles específicos para proteger la plataforma. Entre ellos, consideraría las fallas de control de acceso, evitando que un usuario consulte o modifique órdenes de otros clientes mediante validaciones de permisos en el backend. También protegería la información frente a inyecciones SQL utilizando consultas parametrizadas y validación de entradas.
Implementaría autenticación segura, gestión adecuada de sesiones y almacenamiento protegido de contraseñas mediante algoritmos de hash como Argon2 o bcrypt. Además, utilizaría HTTPS para proteger la comunicación, controlaría los permisos de cada usuario, protegería las credenciales y claves de los servicios externos y validaría las firmas de los webhooks del proveedor de pagos. Finalmente, mantendría las dependencias actualizadas, registraría las operaciones importantes y realizaría auditorías de seguridad para detectar posibles vulnerabilidades.

Incidente

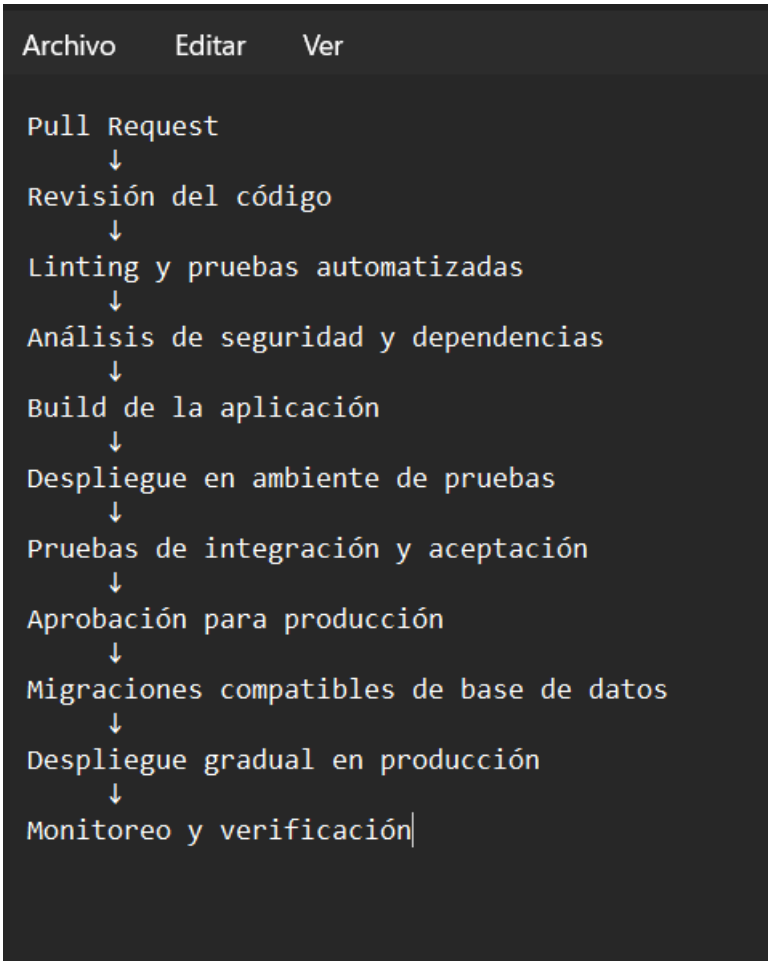
Primero revisaría los registros del backend, los tiempos de respuesta del proveedor de pagos y los identificadores de las solicitudes para identificar en qué momento se están generando las órdenes duplicadas.

También verificaría si el frontend está reenviando solicitudes por tiempos de espera o si el backend está creando una nueva orden cada vez que recibe un intento de compra.
Después comprobaría que el mecanismo de idempotencia esté funcionando correctamente, que cada intento de compra conserve la misma clave y que exista una restricción única en la base de datos para evitar procesamientos duplicados. También revisaría el manejo de los webhooks para asegurar que una misma notificación del proveedor no confirme varias veces la misma orden.
Finalmente, reproduciría el escenario en un ambiente de pruebas simulando respuestas lentas, errores y reintentos. Una vez identificada la causa, corregiría el procesamiento, implementaría controles adicionales y monitorearía las operaciones para confirmar que el problema no vuelva a presentarse.

Datos
Realizaría la migración de forma gradual para evitar interrupciones del servicio y mantener la compatibilidad entre las versiones del backend. Primero agregaría la nueva columna sin eliminar ni modificar inmediatamente la anterior, permitiendo que ambas versiones puedan seguir funcionando.
Posteriormente, desplegaría una versión del backend compatible con las dos columnas y comenzaría a escribir la información en ambas, mientras se realiza la migración de los datos existentes. Después verificaría que la información se haya copiado correctamente y que la nueva columna contenga los valores esperados.
Una vez confirmado que la nueva versión funciona correctamente, cambiaría las lecturas para utilizar la nueva columna y monitorearía el comportamiento de la aplicación. Finalmente, cuando todas las instancias antiguas hayan sido retiradas y se confirme que ya no existe dependencia de la columna anterior, procedería a eliminarla en una migración posterior.

6. DevOps y operación — 5 puntos

33. Dibuje un pipeline CI/CD mínimo desde un pull request hasta producción.



34. Indique dónde ejecutaría linting, pruebas, análisis de dependencias/vulnerabilidades, build y migraciones.
ejecutaría el linting y las pruebas unitarias desde la etapa inicial del Pull Request para detectar errores antes de integrar los cambios. El análisis de dependencias y vulnerabilidades también se realizaría en esta etapa. Posteriormente, ejecutaría el build para comprobar que la aplicación pueda compilarse correctamente. Las pruebas

de integración y E2E se ejecutarían en un ambiente de pruebas. Finalmente, las migraciones se aplicarían de forma controlada durante el despliegue, verificando previamente su compatibilidad con las versiones actuales del backend.

35. Explique una estrategia de rollback.
    implementaría una estrategia de despliegue gradual que permita regresar rápidamente a la última versión estable si se detectan errores en producción. Mantendría disponibles las versiones anteriores de la aplicación y utilizaría mecanismos como Blue-Green Deployment o Canary Release para reducir el riesgo. Si aparecen fallas, redirigiría el tráfico a la versión estable anterior. Para las migraciones de base de datos, priorizaría cambios compatibles y evitaría eliminar columnas o estructuras antiguas hasta confirmar que ninguna versión las necesita.
36. ¿Cómo almacenaría secretos y credenciales?
    almacenaría las credenciales, claves de API, contraseñas de bases de datos y tokens en un servicio especializado de gestión de secretos, como AWS Secrets Manager o AWS Systems Manager Parameter Store. Nunca incluiría estos datos directamente en el código fuente ni en el repositorio. También establecería permisos de acceso limitados, rotación periódica de credenciales y separación de secretos entre los ambientes de desarrollo, pruebas y producción.
37. Mencione métricas y alertas que consideraría críticas para el checkout.
    Monitorearía principalmente el porcentaje de compras exitosas, el tiempo de respuesta del proceso de creación de órdenes, la cantidad de pagos aprobados y rechazados y el número de órdenes duplicadas. También vigilaría los errores del backend, la disponibilidad del proveedor de pagos, los problemas de conexión con la base de datos, los conflictos de inventario y las solicitudes pendientes durante demasiado tiempo. Configuraría alertas automáticas cuando estas métricas superen los límites establecidos, para detectar problemas rápidamente y evitar que afecten la experiencia de compra.

7. Comunicación técnica — 5 puntos

En máximo 250 palabras responda: **“¿Qué decisión de su arquitectura considera más riesgosa, qué alternativa evaluó y qué información necesitaría para confirmar la decisión?”**

la decisión que considero más riesgosa es comenzar CommerceHub con una arquitectura de monolito modular, principalmente porque, a medida que aumenten los usuarios y las operaciones, podría convertirse en un punto de concentración de carga y afectar el rendimiento de toda la plataforma si no se escala adecuadamente.

Como alternativa, evalué implementar una arquitectura de microservicios desde el inicio, separando componentes como inventario, pagos, órdenes y notificaciones. Esta opción permitiría escalar cada servicio de manera independiente, pero también aumentaría la complejidad de desarrollo, despliegue, comunicación entre servicios y monitoreo.

Mi decisión inicial sería mantener el monolito modular, procurando que cada módulo tenga responsabilidades bien definidas y pueda separarse en el futuro si las necesidades del negocio lo requieren.

Para confirmar esta decisión necesitaría conocer información más precisa sobre el crecimiento esperado de usuarios, el volumen de transacciones por segundo, los picos de concurrencia, la cantidad de órdenes procesadas y los requerimientos de disponibilidad. También sería importante conocer el presupuesto de infraestructura, el tamaño del equipo de desarrollo y las proyecciones de crecimiento del negocio.

Formato de entrega del candidato

6. Documento PDF/Markdown o repositorio con respuestas organizadas por sección.
7. Diagrama de arquitectura legible.
8. Código o pseudocódigo ejecutable/concreto para los ejercicios 1 y 3.
9. Supuestos claramente declarados. Si falta información, el candidato debe indicar el supuesto adoptado y su impacto.
10. Se valora más una solución simple y defendible que una solución sobredimensionada.